

Saphul PMS

1) USERS

Purpose:

Stores all system users (Admin, PMs, Members, Clients, QA, etc.)

Use:

Authentication & user identity only.

Column	Type	Description
id	INT (PK)	Unique user ID
name	VARCHAR(100)	Full name
email	VARCHAR(150) UNIQUE	Login email
password	VARCHAR	Encrypted password
job_title	VARCHAR(50)	Developer / Analyst / Marketer
department	VARCHAR(50)	Engineering / Marketing / Analytics
status	ENUM	ACTIVE / INACTIVE
is_deleted	BOOLEAN	Soft delete
deleted_at	TIMESTAMP	Deletion time
created_at	TIMESTAMP	Created time
updated_at	TIMESTAMP	Updated time

2)ROLES

Purpose:

Defines user roles dynamically (no ENUM).

Use:

Groups permissions logically.

Column	Type	Description
id	INT (PK)	Role ID
name	VARCHAR(50) UNIQUE	Role name (ADMIN, CLIENT, QA, etc.)
created_at	TIMESTAMP	Created time

3) USER_ROLES

Purpose:

Assigns roles to users.

Use:

Allows one user to have multiple roles.

Column	Type	Description
user_id	INT (FK)	User
role_id	INT (FK)	Role
PK	(user_id, role_id)	Composite PK

4) PERMISSIONS

Purpose:

Defines system actions.

Use:

Controls what actions are allowed in the system.

Column	Type	Description
id	INT (PK)	Permission ID
code	VARCHAR(50) UNIQUE	Action code (TASK_EDIT, TASK_APPROVE)

5) ROLE_PERMISSIONS

Purpose:

Maps which role can perform which action.

Use:

Defines access rules without code changes.

Column	Type	Description
role_id	INT (FK)	Role
permission_id	INT (FK)	Permission
PK	(role_id, permission_id)	Composite PK

6) PROJECTS

Use:

Stores projects, tracks lifecycle, enforces Admin approval for completion, supports Waterfall / Agile / Hybrid.

Column	Type	Description
id	INT (PK)	Project ID
name	VARCHAR(150)	Project name
description	TEXT	Project details
methodology	ENUM	WATERFALL / AGILE / HYBRID
status	ENUM	PENDING / IN_PROGRESS / COMPLETED / DELAYED / CLOSED
review_status	ENUM	PENDING / APPROVED / REJECTED
start_date	DATE	Start date
end_date	DATE	Planned end date
review_requested_at	TIMESTAMP	Sent for admin approval
reviewed_at	TIMESTAMP	Approval time
reviewed_by	FK → users.id	Admin approver
created_by	FK → users.id	Admin creator
is_deleted	BOOLEAN	Soft delete
deleted_at	TIMESTAMP	Deletion time
created_at	TIMESTAMP	Created time
updated_at	TIMESTAMP	Updated time

7) SPRINTS

Use:

Lightweight agile support (only for AGILE / HYBRID projects).

Column	Type	Description
id	INT (PK)	Sprint ID
project_id	FK → projects.id	Parent project
name	VARCHAR(100)	Sprint name
start_date	DATE	Sprint start
end_date	DATE	Sprint end
status	VARCHAR(20)	PLANNED / ACTIVE / COMPLETED

8) PROJECT_MEMBERS

Use:

Defines which users belong to which projects and in what project-level role.

Column	Type	Description
id	INT (PK)	Row ID
project_id	FK → projects.id	Project
user_id	FK → users.id	User
role	ENUM	PROJECT_MANAGER / MEMBER
assigned_at	TIMESTAMP	Assignment time
is_deleted	BOOLEAN	Soft removal
deleted_at	TIMESTAMP	Removal time

9)TASKS (Billing Optional)

Use:

Stores tasks, approvals, backlog, subtasks, sprint mapping.

Billing fields are **optional** and **internal only**.

Column	Type	Description
id	INT (PK)	Task ID
project_id	FK → projects.id	Parent project
sprint_id	FK → sprints.id (NULL)	Sprint (optional)
parent_task_id	FK → tasks.id (NULL)	Subtask
backlog_order	INT	Backlog order
title	VARCHAR(150)	Task title
description	TEXT	Task details
assigned_to	FK → users.id	Assigned member
created_by	FK → users.id	Admin / PM
status	ENUM	TODO / IN_PROGRESS / BLOCKED / DONE / CANCELLED
review_status	ENUM	NONE / UNDER_REVIEW / APPROVED / REJECTED
approval_status	ENUM	NONE / PENDING / APPROVED / REJECTED
priority	ENUM	LOW / MEDIUM / HIGH
start_date	DATE	Start date
due_date	DATE	Deadline
estimated_hours	DECIMAL(6,2)	Planned effort (optional)
actual_hours	DECIMAL(6,2)	Derived from time logs
billable	BOOLEAN	DEFAULT false

billing_rate	DECIMAL(10,2)	NULL if non-billable
review_requested_at	TIMESTAMP	Sent for PM review
reviewed_at	TIMESTAMP	Approved time
reviewed_by	FK → users.id	Project Manager
completed_at	TIMESTAMP	Final completion
is_deleted	BOOLEAN	Soft delete
deleted_at	TIMESTAMP	Deletion time
created_at	TIMESTAMP	Created
updated_at	TIMESTAMP	Updated

10) TIME_LOGS

Use:

Tracks who worked, when, and how long.

Used for productivity and (optional) billing.

Column	Type	Description
id	INT (PK)	Time log ID
task_id	FK → tasks.id	Task
user_id	FK → users.id	Worker
log_date	DATE	Work date
hours	DECIMAL(5,2)	Hours worked
description	TEXT	Work notes
created_at	TIMESTAMP	Logged time

11) FILES

Use:

Stores file metadata only.

Actual files live in cloud storage.

Column	Type	Description
id	INT (PK)	File ID
project_id	FK → projects.id	Project
task_id	FK → tasks.id (NULL)	Task (optional)
uploaded_by	FK → users.id	Uploader
file_name	VARCHAR(255)	File name
file_type	VARCHAR(50)	pdf / docx / png
file_url	TEXT	Storage path
version	VARCHAR(20)	v1 / v2 / v3
is_latest	BOOLEAN	Latest version
is_deleted	BOOLEAN	Soft delete
deleted_at	TIMESTAMP	Deletion time
created_at	TIMESTAMP	Uploaded

12)ACTIVITY_LOGS

Use:

Audit trail for all important actions.

Column	Type	Description
id	INT (PK)	Log ID
entity_type	ENUM	PROJECT / TASK / FILE
entity_id	INT	Entity ID
action	VARCHAR(50)	created / updated / approved
performed_by	FK → users.id	Actor
created_at	TIMESTAMP	Action time

13)APPROVALS

Use:

Centralized approval tracking (projects & tasks).

No payment logic.

Column	Type	Description
id	INT (PK)	Approval ID
entity_type	ENUM	PROJECT / TASK
entity_id	INT	Entity ID
requested_by	FK → users.id	Requester
requested_at	TIMESTAMP	Request time
approved_by	FK → users.id	Approver
approved_at	TIMESTAMP	Approval time
status	ENUM	PENDING / APPROVED / REJECTED
remarks	TEXT	Comments

14) PROJECT_TASK_STATS – Final Table Format

Column	Type	Description
project_id	INT (PK, FK → projects.id)	Project
total_tasks	INT	Total tasks
completed_tasks	INT	Completed tasks
blocked_tasks	INT	Blocked tasks
overdue_tasks	INT	Overdue tasks
billable_hours	DECIMAL(8,2)	Total billable hours
non_billable_hours	DECIMAL(8,2)	Total non-billable hours
last_updated	TIMESTAMP	Last sync time

My Learnings from Database V1 to V2

- Separated **user identity, roles, and permissions** instead of mixing them in one table.
- Moved from **role-based checks to permission-based access**, making the system scalable.
- Avoided using ENUMs for roles to support future roles without database changes.
- Introduced a **single TASKS table** to handle backlog, sprints, subtasks, and approvals.
- Added **TIME_LOGS** to properly track who worked, when, and for how long.
- Made **billing optional and internal**, keeping clients away from financial logic.
- Centralized approvals using an **APPROVALS** table for better auditability.
- Restricted project visibility using **PROJECT_MEMBERS** to improve data security.